the default template setup you saw in listing 23.1, but make sure to reference your existing `AppDbContext`.

Listing 23.3 Adding ASP.NET Core Identity services to the recipe app

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>            ❶
    options.UseSqlite(builder.Configuration                       ❶
        .GetConnectionString("DefaultConnection")!));             ❶

builder.Services.AddDefaultIdentity<ApplicationUser>(options =>   ❷
        options.SignIn.RequireConfirmedAccount = true)            ❷
    .AddEntityFrameworkStores<AppDbContext>();                    ❸

builder.Services.AddRazorPages();
builder.Services.AddScoped<RecipeService>();
```

❶ The existing service configuration is unchanged.

❷ Adds the Identity services to the DI container and uses a custom user type, ApplicationUser

❸ Makes sure you use the name of your existing DbContext app

This adds all the necessary services and configures Identity to use EF Core. I've introduced a new type here, `ApplicationUser`, which we'll use to customize our user entity later. You'll see how to add this type in section 23.4.2.

The next step is optional: add the `AuthenticationMiddleware` after the call to `UseRouting()` on `WebApplication`, as shown in the following listing. As I mentioned previously, the authentication middleware is added automatically by `WebApplication`, so this step is optional.

Listing 23.4 Adding `AuthenticationMiddleware` to the recipe app

```
app.UseStaticFiles();          ❶

app.UseRouting();

app.UseAuthentication();       ❷
app.UseAuthorization();        ❸
```